

MemOS-Q: A Self-Evolving Memory Operating System for AI Agents

Built with QwenCloud

MemOS-Q is a production-oriented memory layer that enables AI agents to remember, reason, adapt, and self-correct across conversations, documents, images, and external tools.

Unlike traditional chatbots that rely solely on context windows, MemOS-Q introduces persistent memory with confidence scoring, conflict resolution, multimodal understanding, explainable recall, and autonomous memory maintenance.

The system acts as a Memory Operating System for AI applications, allowing agents to continuously learn from interactions while remaining transparent, auditable, and user-controlled.

Problem

Modern AI assistants suffer from three major limitations:

1. No long-term memory
2. Information is forgotten between sessions.
3. Users repeatedly provide the same context.
4. Memory hallucinations
5. Existing memory systems often store incorrect or outdated facts.
6. There is little transparency into why something was remembered.
7. Lack of multimodal continuity
8. Information contained in screenshots, PDFs, presentations, repositories, and documents is typically disconnected from future conversations.

As AI agents become increasingly autonomous, memory management becomes a critical infrastructure problem.

Solution

MemOS-Q introduces four core innovations:

Explainable Memory

Every recalled memory includes:

- source session
- confidence score
- timestamp
- reasoning path

The agent can explain why a memory was retrieved.

Self-Correcting Memory

The system automatically detects:

- contradictions
- outdated information
- superseded preferences

and updates memory state accordingly.

Multimodal Memory

Using Qwen3-VL, the system can remember information extracted from:

- PDFs
- screenshots
- slide decks
- images
- diagrams
- whiteboard photos

Memory is no longer limited to chat messages.

Autonomous Memory Maintenance

Dedicated maintenance agents continuously:

- merge duplicates

- summarize old conversations
- promote stable facts
- archive stale information
- update confidence scores

The memory base improves over time instead of growing uncontrollably.

Why It Matters

MemOS-Q transforms AI from:

"Stateless chatbot"

into

"Persistent personalized assistant"

with applications in:

- personal AI assistants
 - education
 - healthcare support systems
 - enterprise copilots
 - customer support
 - developer tooling
 - autonomous agents
-

Architecture

High-Level Architecture

User Input

↓

Orchestrator Agent

↓

Memory Pipeline

└── Retrieval Agent

└── Memory Agent

└── Profile Agent

└── Audit Agent

└── Compaction Agent

↓

Qwen Models

↓

Response Generation

↓

Memory Write-Back

QwenCloud Usage

This project is deeply integrated with QwenCloud services.

Qwen3.5-Plus

Primary reasoning model.

Responsibilities:

- response synthesis
- long-context reasoning
- memory decisions
- profile inference
- multi-turn dialogue

Qwen3.5-Flash

High-speed helper model.

Responsibilities:

- intent classification
 - memory extraction
 - memory scoring
 - deduplication
 - routing
 - summarization
-

Qwen3-VL-Plus

Multimodal memory engine.

Responsibilities:

- screenshot understanding
 - PDF extraction
 - slide analysis
 - image memory creation
 - document summarization
-

Qwen Batch API

Offline processing.

Responsibilities:

- nightly memory compaction
 - clustering
 - memory cleanup
 - profile consolidation
-

Qwen-Agent

Agent orchestration framework.

Used to implement:

- tool calling
- memory workflows
- MCP integrations
- multi-agent coordination

MCP Integrations

MemOS-Q supports Model Context Protocol integrations.

Examples:

- GitHub MCP
- Notion MCP
- Google Drive MCP
- Calendar MCP

This allows memory generation from external knowledge sources.

Example:

User uploads a project proposal in Notion.

↓

Qwen3-VL extracts project goals.

↓

Memory is generated.

↓

Future conversations automatically use project context.

Memory Architecture

Layer 1 — Working Memory

Short-term context.

Stores:

- active conversation
- current tasks
- temporary state

Lifetime:

single session

Layer 2 — Episodic Memory

Session-based events.

Examples:

- User requested vegetarian recipes
 - User is preparing for a hackathon
 - User uploaded a system design document
-

Layer 3 — Semantic Memory

Stable long-term facts.

Examples:

- Prefers concise responses
 - Uses Python
 - Interested in AI agents
-

Layer 4 — Operational Memory

Agent-specific knowledge.

Examples:

- failed tool calls
 - previous retrieval attempts
 - safety decisions
 - execution traces
-

Memory Graph

Unlike traditional vector-only memory systems, MemOS-Q stores relationships between memories.

Node Types:

- preference

- fact
- project
- event
- task

Relationship Types:

- derived_from
- contradicts
- supersedes
- related_to
- supports

Example:

Uses Python



Builds AI Agents



Participates in Hackathons



Interested in Qwen

This graph improves retrieval quality and explainability.

Memory Quality Engine

Every memory receives:

Importance Score

Measures long-term usefulness.

Confidence Score

Measures certainty.

Novelty Score

Measures uniqueness.

Stability Score

Measures persistence across sessions.

Example:

```
{ "importance": 0.91, "confidence": 0.95, "novelty": 0.74, "stability": 0.83 }
```

Conflict Resolution

Example:

Old Memory

Uses MacBook M3

New Memory

Uses ThinkPad X1

System automatically:

- detects contradiction
- deprecates old memory
- activates new memory
- records audit trail

This prevents memory drift.

Explainable Recall

Every retrieved memory contains provenance.

Example:

"I remember that you build FastAPI applications."

Source:

Session #12

Confidence: 94%

Last Verified: June 2026

This improves trust and transparency.

Retrieval Engine

Hybrid retrieval combines multiple ranking signals.

Final Score

=

Semantic Similarity

+

Keyword Match

+

Recency

+

Importance

+

Confidence

+

Graph Proximity

Example:

score = 0.35 semantic + 0.20 keyword + 0.15 recency + 0.15 importance + 0.10 confidence + 0.05 graph

This outperforms vector-only retrieval.

Autonomous Memory Compaction

Scheduled maintenance jobs continuously improve memory quality.

Functions:

Duplicate Merging

Combines redundant memories.

Semantic Promotion

Frequently repeated facts become profile memories.

Confidence Decay

Unverified memories gradually lose weight.

Archival

Unused memories move to cold storage.

User Control

Users maintain full control.

Supported actions:

- remember this
- forget this
- update memory
- inspect memory
- view memory graph

All changes are logged.

Database Design

memories

- id
- user_id

- memory_type
 - content
 - embedding
 - confidence_score
 - importance_score
 - novelty_score
 - stability_score
 - status
 - version
 - created_at
 - last_recalled_at
 - expires_at
-

memory_edges

- id
 - source_memory
 - target_memory
 - relation_type
-

user_profile

- user_id
 - preferences
 - interests
 - communication_style
-

audit_log

- action
 - actor
 - previous_value
 - new_value
 - timestamp
-

Technology Stack

Frontend

- Next.js

- Tailwind CSS
- shadcn/ui
- React Flow

Backend

- FastAPI
- Qwen-Agent

Models

- Qwen3.5-Plus
- Qwen3.5-Flash
- Qwen3-VL-Plus
- Qwen Batch API

Storage

- PostgreSQL
- pgvector
- Redis
- S3-compatible Object Storage

Background Jobs

- Celery

Monitoring

- Langfuse
- OpenTelemetry
- Prometheus
- Grafana

Deployment

- Docker

Innovation

MemOS-Q introduces:

- Explainable memory retrieval
- Memory confidence scoring
- Graph-linked memory relationships
- Multimodal memory ingestion
- Autonomous memory maintenance

- Conflict-aware memory updates
 - User-auditable memory lifecycle
-

Impact

MemOS-Q can serve as the memory layer for:

- AI copilots
- educational tutors
- enterprise assistants
- autonomous agents
- developer agents
- customer support systems

The architecture is model-agnostic and designed for large-scale deployment.

Future Work

- Cross-agent shared memory
 - Federated memory synchronization
 - Team knowledge graphs
 - Long-term behavioral adaptation
 - Agent-to-agent memory exchange
 - Reinforcement-based memory ranking
-

Built for Qwen Code Challenge

MemOS-Q demonstrates advanced usage of:

- Qwen3.5-Plus
- Qwen3.5-Flash
- Qwen3-VL-Plus
- Qwen Batch API
- Qwen-Agent
- MCP integrations

while solving a fundamental challenge in next-generation AI systems:

How can AI remember responsibly, transparently, and at scale?